



Università
Ca'Foscari
Venezia

Heuristic Search

FOUNDATIONS OF ARTIFICIAL INTELLIGENCE

Andrea Torsello

Dip. Sc. Ambientali, Informatica, Statistica
Università Ca'Foscari Venezia

Heuristics

The assumption behind **blind search** is that we have no way of telling whether a particular search direction is likely to lead us to the goal or not

The key idea behind **informed or heuristic** search algorithms is to exploit a task-specific measure of goodness to try to either reach the goal more quickly or find a more desirable goal state.

Heuristic: From the Greek for “find”, “discover”.

Heuristics are criteria, methods, or principles for deciding which among several alternative courses of action promises to be the most effective in order to achieve some goal”.

Judea Pearl “ Heuristics” 1984

Heuristics



Problem knowledge comes in the form of an heuristic function $h'(n)$

- ▶ $h'(n)$ assigns to each node n an estimate of the optimal path length from n to a goal node
- ▶ We assume that h' is non-negative and that $h'(n)=0$ iff n is a goal node.

It is a technique designed to solve a problem quickly

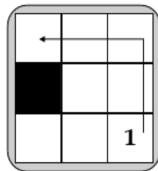
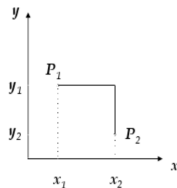
Heuristics for the game of 8

$h'1(n)$ = number of misplaced tiles

$h'2(n)$ = sum of Manhattan distance* of each tile

* Given two planar points of coordinate $P1 = (x_1, y_1)$ and $P2 = (x_2, y_2)$, the Manhattan distance is defined as

$$D(P_1, P_2) = |x_1 - x_2| + |y_1 - y_2|$$



Heuristics for the game of 8

Example

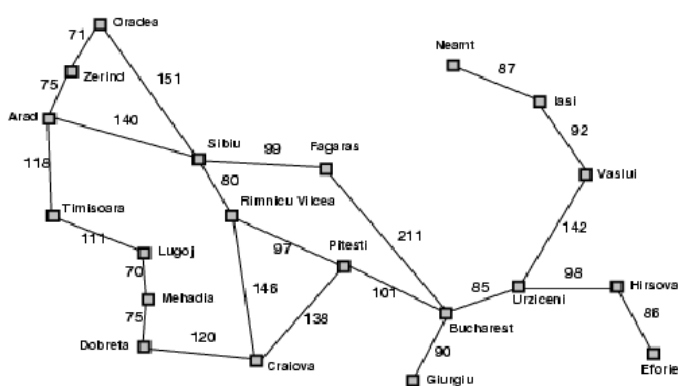
$$h'1(n) = 7$$

$$h'2(n) = 4+0+2+3+1+3+1+3=17$$

6	2	8
	3	5
4	7	1

Travel Planning

$h'(n)$ = straight-line distance between n and goal node



Straight-line distance
to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Best First (Greedy) Search

The idea behind best first search is to always explore the most promising path first.

Can be obtained from the generic search stab function by selecting from L the node with the minimal value of the expected distance to goal (value of heuristic function)

Best First (Greedy) Search

Example (Best First (Greedy) Search)

```
search(G,source,heuristic,is_goal):  
    L=[source] # list of visited but  
               # not expanded nodes  
    while L:  
        n = extract_min(L,heuristic)  
        # expand node with minimum heuristic  
  
        if is_goal(n): # goal node found  
            return path(source,n)  
  
        # goal not found, visit adjacent nodes  
        children = G.adjacent(n)  
        insert_into(L, children) # anywhere  
    return FAIL # goal not present in G
```

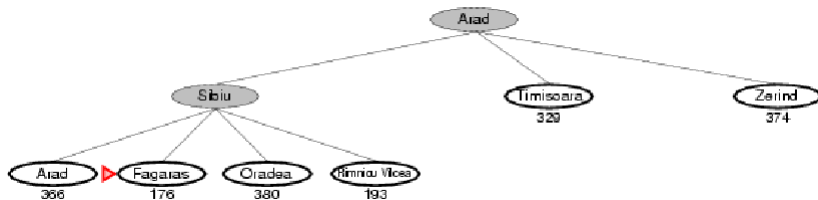
Best First Search Example



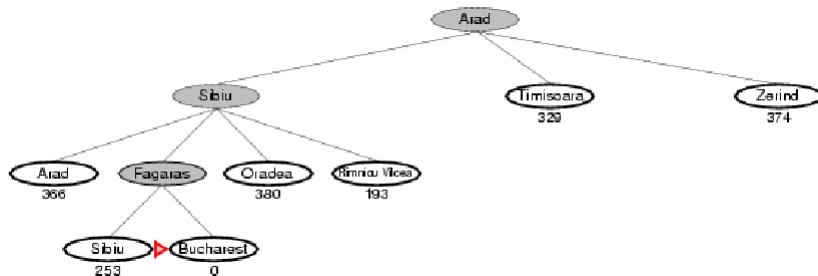
Best First Search Example



Best First Search Example



Best First Search Example



Analysis of Best First

Complete? No – can get stuck in loops, e.g., lasi $\rightarrow$ Neamt $\rightarrow$ lasi $\rightarrow$ Neamt

Time? $O(b^m)$, but a good heuristic can give dramatic improvement

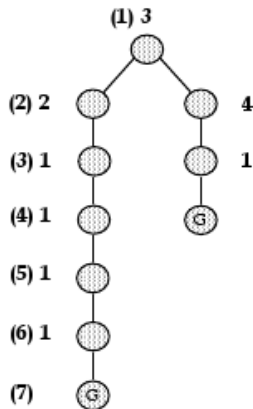
Space? $O(b^m)$ – keeps all nodes in memory

Optimal? No

Example of non optimality

The number in parentheses represent the order of expansion

The other the value of the heuristic function h'



A* algorithm mixes the optimality of uniform cost with the heuristic search of best first

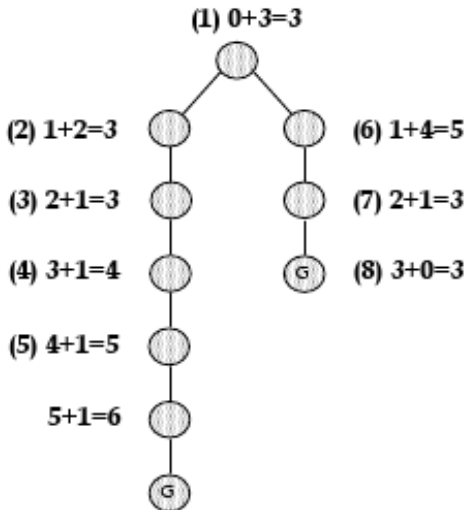
A* realizes a best first search with evaluation function

$$f(n) = g(n) + h'(n)$$

- ▶ $g(n)$ is the path length from the root to n
- ▶ $h'(n)$ is the heuristic prediction of the cost from n to the goal

Example (A*)

```
search(G,source,heuristic,is_goal):  
    # list of visited nodes with their evaluation  
    L=[(source,heuristic(source))]  
    while L:  
        # expand node with minimum evaluation  
        n,d = extract_min(L)  
  
        if is_goal(n): # goal node found  
            return path(source,n)  
  
        # goal not found, visit adjacent nodes  
        for v in G.adjacent(n)  
            insert_into(L, (v,d+1+heuristic(v)))  
    return FAIL # goal not present in G
```



A*

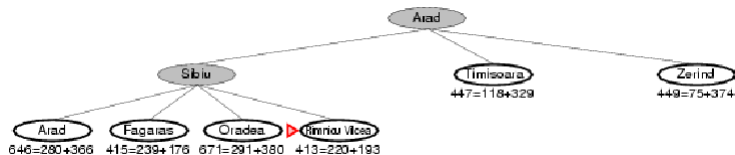
Search Example

▶ Aiad
366=0+366

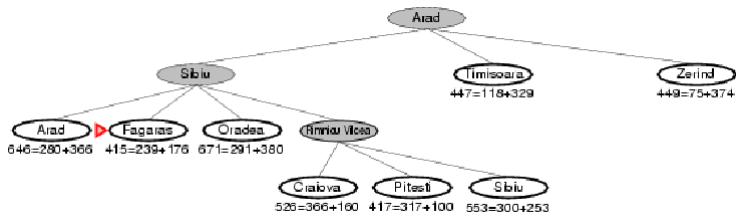
A* Search Example



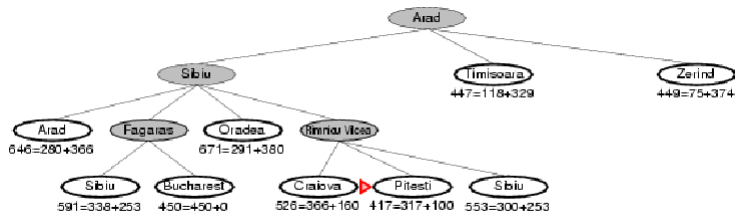
A* Search Example



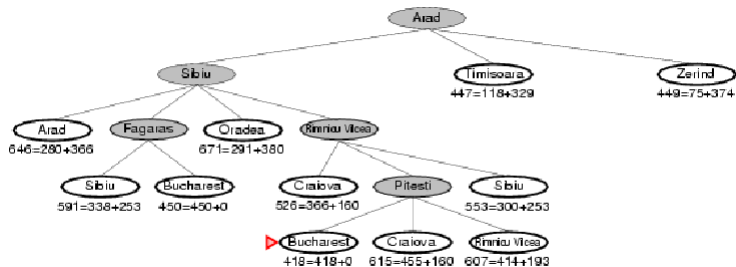
A* Search Example



A* Search Example



A* Search Example



Consistent heuristics

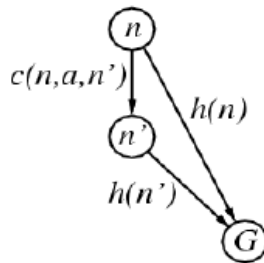
A heuristic is **consistent** if for every node n , every successor n' of n generated by any action a , with cost $c(n, a, n')$

$$h'(n) \leq c(n, a, n') + h'(n')$$

If h' is consistent, we have

$$\begin{aligned} f(n') &= g(n') + h'(n') \\ &= g(n) + c(n, a, n') + h'(n') \\ &\geq g(n) + h'(n) = f(n) \end{aligned}$$

$f(n)$ is **non-decreasing** along any path



Corollary: $h'(n) \leq h(n)$, where $h(n)$ is the real (unknown) distance from the goal

Optimality of A^*

Theorem: If function h' is consistent, then A^* is optimal

Proof: Let s be an optimal goal node and x a non-optimal goal.
Let $n_0, n_1, \dots, n_k = s$ be the path from the root to s . For all
 $i = 0 \dots k$, we have $f(n_i) < f(x)$.

In fact:

$$\begin{aligned} f(n_i) &= g(n_i) + h'(n_i) \leq g(n_i) + h(n_i) = g(s) \\ &< g(x) = g(x) + h'(x) = f(x) \end{aligned}$$

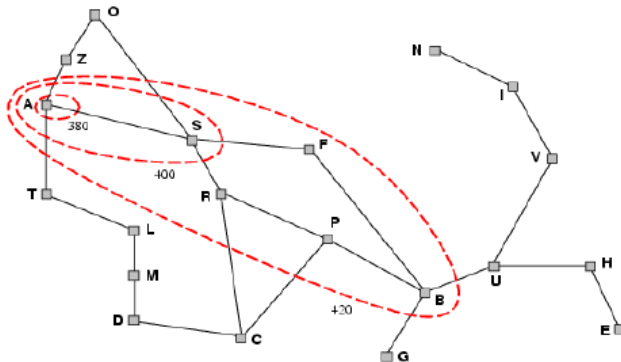
But then s must be expanded before x .

Optimality of A*

A* expands nodes in order of increasing f value

Gradually adds " f -contours" of nodes

Contour i has all nodes with $f = f_i$, where $f_i < f_{i+1}$



Properties of A*

Complete? Yes (unless there are infinitely many nodes with $f \leq f(G)$)

Time? Exponential

Space? Exponential (Keeps all nodes in memory)

Optimal? Yes

A* is an Heuristic counterpart of breadth first. We can improve the space requirements in a way similar to iterative deepening

Iterative Deepening A*

(IDA*)

Example (IDA*)

```
iterative_deepening(G,source,heuristic,is_goal):  
    C=0  
    while C<max_depth:  
        # list of visited nodes with their f  
        L=[(source,heuristic(source))]  
        while L:  
            n,f = L.pop_left()  
            if is_goal(n):  
                return path(source,n)  
            # visit adjacent nodes if f<C  
            for v in G.adjacent(n)  
                fv=depth(v)+heuristic(v)  
                if fv<=C:  
                    L.append((v,fv))  
            C+=1 # increase C and restart  
    return FAIL
```

IDA* Example

